COM 3401
DATA COMMUNICATION AND NETWORKING

Lecture 09

FRAMING

P. Keerthishan
Temporary Assistant Lecturer
Dept. of Computing
Faculty of Applied Sciences, RUSL

Topics covered

- Introduction to Framing
- Character Count
- Character / Byte Stuffing
- Bit Stuffing
- Comparison

Reference:

William Stallings, *Data and Computer Communications*, 8th Edition

Andrew S. Tanenbaum, *Computer Networks,* 5th Edition

Kurose and Ross, Computer Networking – A Top-Down Approach, 6th Edition

9.1 Introduction to Framing



Framing

- The data link layer receives a continuous stream of bits from the physical layer.
- It divides this bit stream into **discrete frames**.
- The receiver must determine:
  - Where a frame **starts**
  - Where a frame **ends**

- **Framing methods covered:**
  1. Character Count
  2. Character/Byte Stuffing
  3. Bit Stuffing

9.2
Character Count

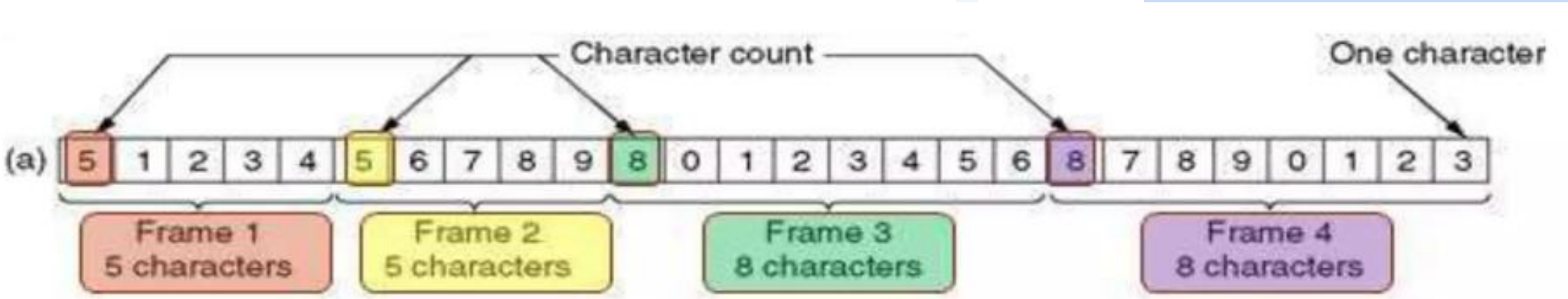


Character Count Framing

Character Count

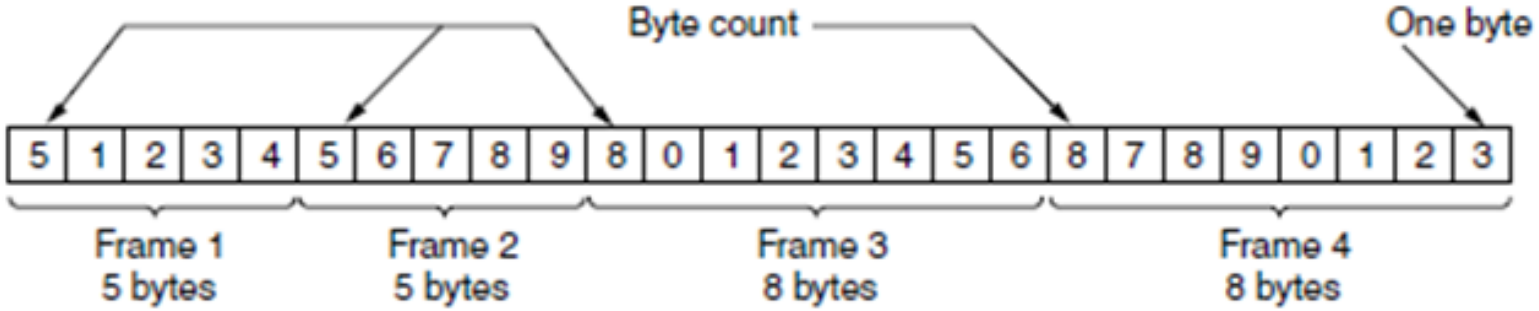
- A field in the frame header specifies the **number of bytes in the frame**.
- The receiver reads the count.
- It then knows how many bytes belong to the frame.
- Therefore, the count identifies the **end of the frame**.

Basic idea:



Character Count: Example

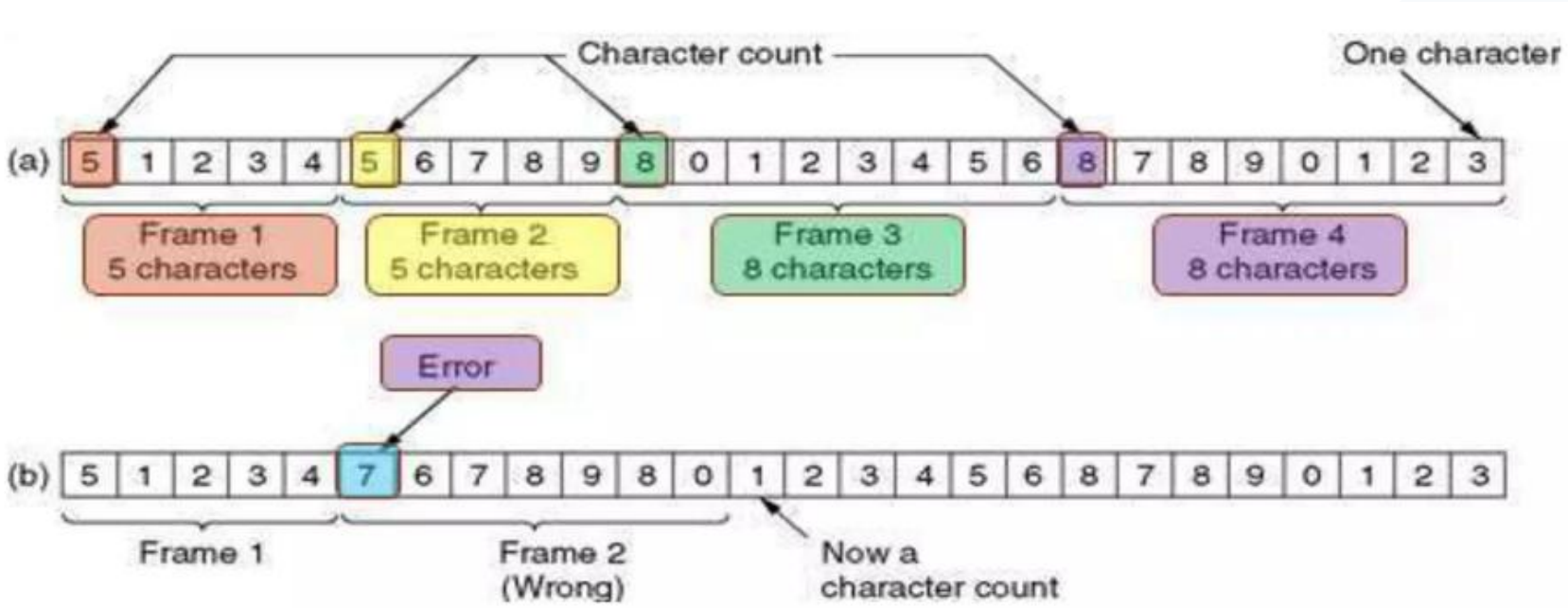
- Suppose four frames contain:
  - Frame 1 → 5 bytes
  - Frame 2 → 5 bytes
  - Frame 3 → 8 bytes
  - Frame 4 → 8 bytes
- The transmitted stream contains the corresponding count values:



- **Key point**
  - The receiver uses the **count field** to locate the end of each frame

Problem with Character Count

- Example:



- The receiver now reads the wrong number of bytes.
- **Result:**
  - Receiver loses synchronization.
  - It cannot correctly identify the beginning of the next frame.
  - Even if an error is detected later, the receiver does not know where the next frame begins.

Character Count: Evaluation

Advantages

- Simple mechanism
- Little framing overhead
- Receiver can determine frame length directly from the count

Major disadvantage

- **A corrupted count can destroy synchronization.**
  - Therefore, **rarely used by itself**.

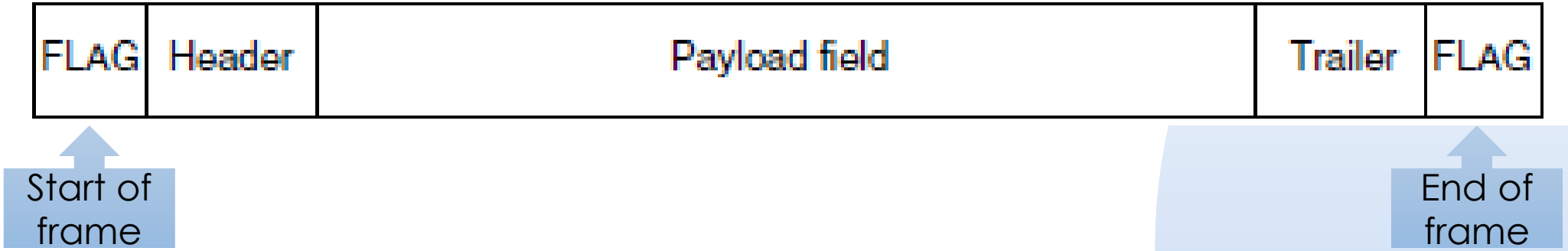
- **Main idea:**
  If the count is wrong, the receiver may not know where the next frame starts.

9.3 Character / Byte Stuffing



Flag Bytes

- Instead of counting bytes, a frame can be marked using **special bytes**.



- A special byte is called a **flag byte.**
- The same flag can be used at both ends.
- Two consecutive flag bytes can indicate:
  - End of one frame
  - Start of the next frame

The Problem with Flag Bytes

What if the **FLAG byte occurs inside the data**?

FLAG	Header	Data	FLAG	Data	Trailer	FLAG

- The receiver cannot automatically know whether the flag byte:
  - Delimits a frame, or
  - Is actually part of the data.
- This can interfere with framing.
- **Example**
  - Binary data such as:
  - Photographs
  - Songs
- may contain the same byte value as the flag.

Byte Stuffing

- When the flag byte occurs inside the data:
  - ➡️ Insert a special **ESC (escape) byte** before it.
- Original Data:

A	B	FLAG	C

- After stuffing:

A	B	ESC	FLAG	C

- The receiver removes the ESC byte during **destuffing**.

A	B	ESC	FLAG	C

➡️ Destuffing ➡️

A	B	FLAG	C

What if ESC Occurs in the Data?

The escape byte itself must also be protected.

- If, data:

A	B	ESC	C

- Then transmit:

A	B	ESC	ESC	C

- At the receiver:

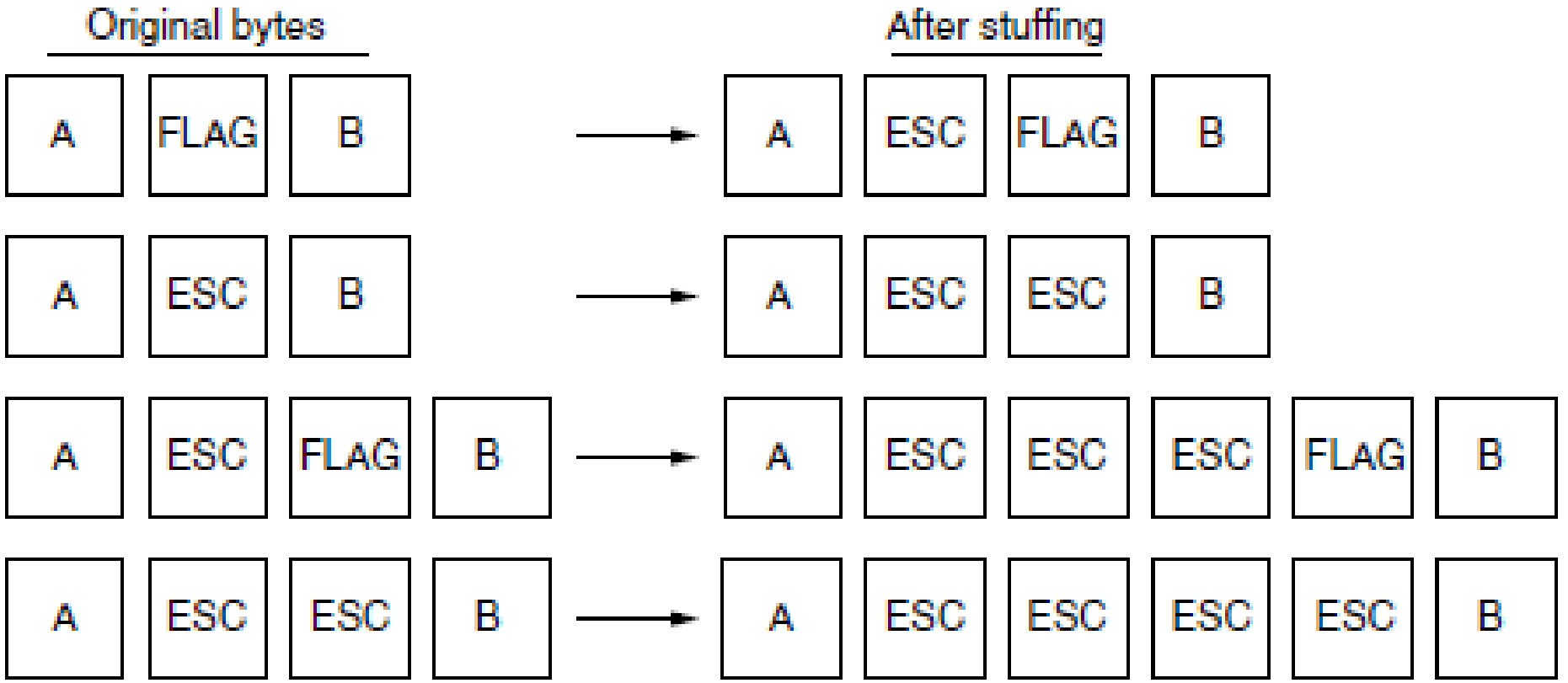
ESC	ESC	➡️	ESC

Rule

- **Stuff an ESC before:**
  - A FLAG byte occurring in the data
  - An ESC byte occurring in the data

Framing

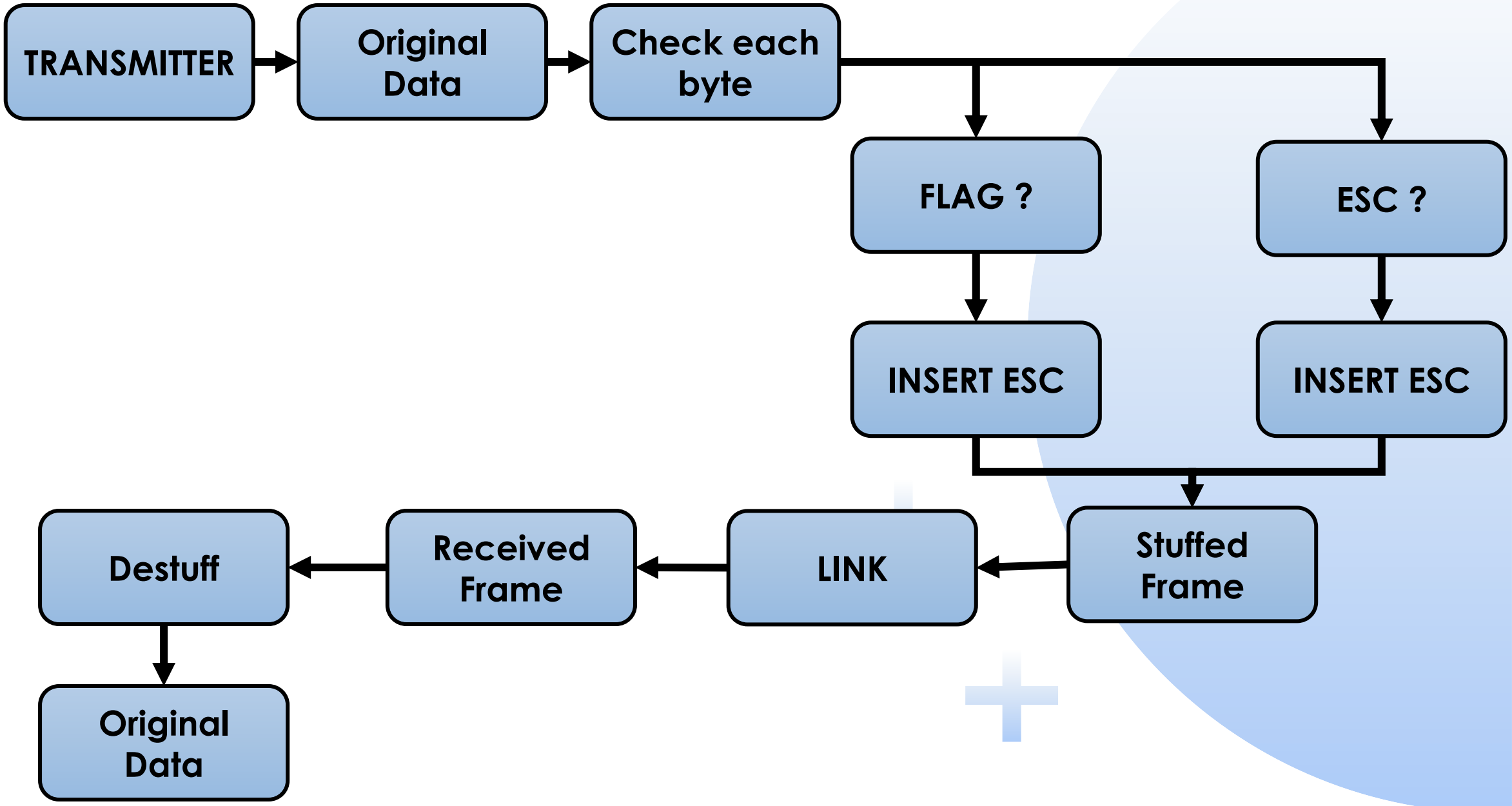
- Byte Stuffing Examples



Key observation

- After **destuffing**, the receiver obtains exactly the original byte sequence.

Byte Stuffing: Complete Process



9.4
Bit Stuffing

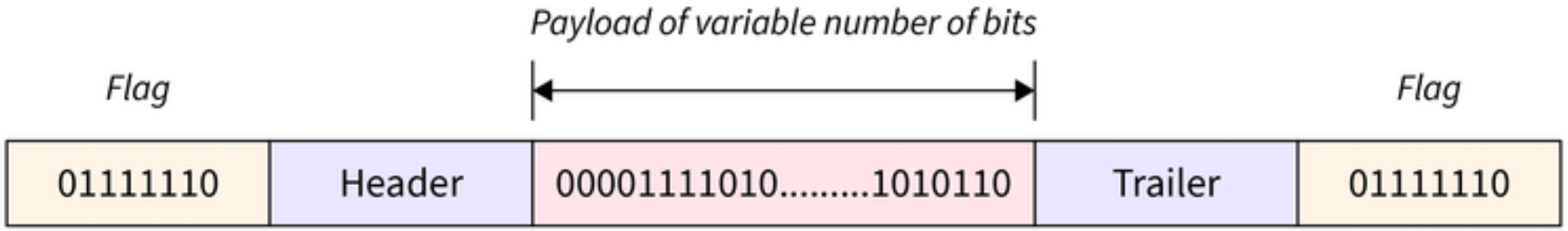


Why Bit Stuffing?

- Byte stuffing is tied to **byte-oriented data**.
- Bit stuffing allows framing at the **bit level**.
- Frame boundaries are represented by a special bit pattern.
- The HDLC flag is:

01111110

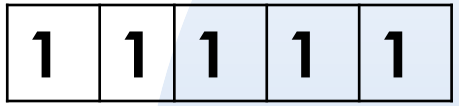
- The flag marks the beginning and end of a frame.



Bit Stuffing Rule

Rule

At Sender:

- Whenever the transmitter encounters: 

  - i.e.: **five consecutive 1s**
- it automatically inserts: **0**

- So:



Bit Stuffing Rule

At the receiver:

- When the receiver sees: | 1 | 1 | 1 | 1 | 1 | 0 |
- it removes the inserted **0**:

1	1	1	1	1	0	➡️	1	1	1	1	1

- **Key idea:**
  - **Sender → stuff 0**
  - **Receiver → destuff 0**

Bit Stuffing Example

Original pattern:

```
11111111111101111110111111110
```

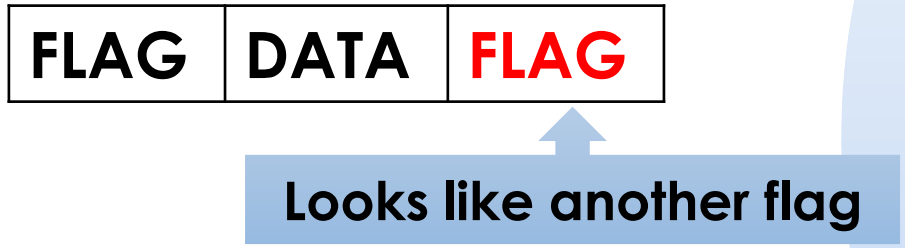
After bit-stuffing:

```
111110111111011011111010111111010
```

- The inserted **0**s prevent the data from forming the flag pattern.

Bit Stuffing with the HDLC Flag

- **HDLC Flag: 01111110**
- The problem:
  - Data contains: **01111110**
- Without bit stuffing:



- With bit stuffing:



- The data can therefore contain the flag pattern without being mistaken for a frame boundary.

Bit Destuffing at the Receiver

The receiver continuously examines the incoming bits.

When it sees: | 1 | 1 | 1 | 1 | 1 | 0 |

the **0** is removed.

| 1 | 1 | 1 | 1 | 1 | 0 | ➡ | 1 | 1 | 1 | 1 | 1 |

But:

| 1 | 1 | 1 | 1 | 1 | 1 | 0 |

is recognized as the **flag pattern** when considered with the preceding bit.

In HDLC

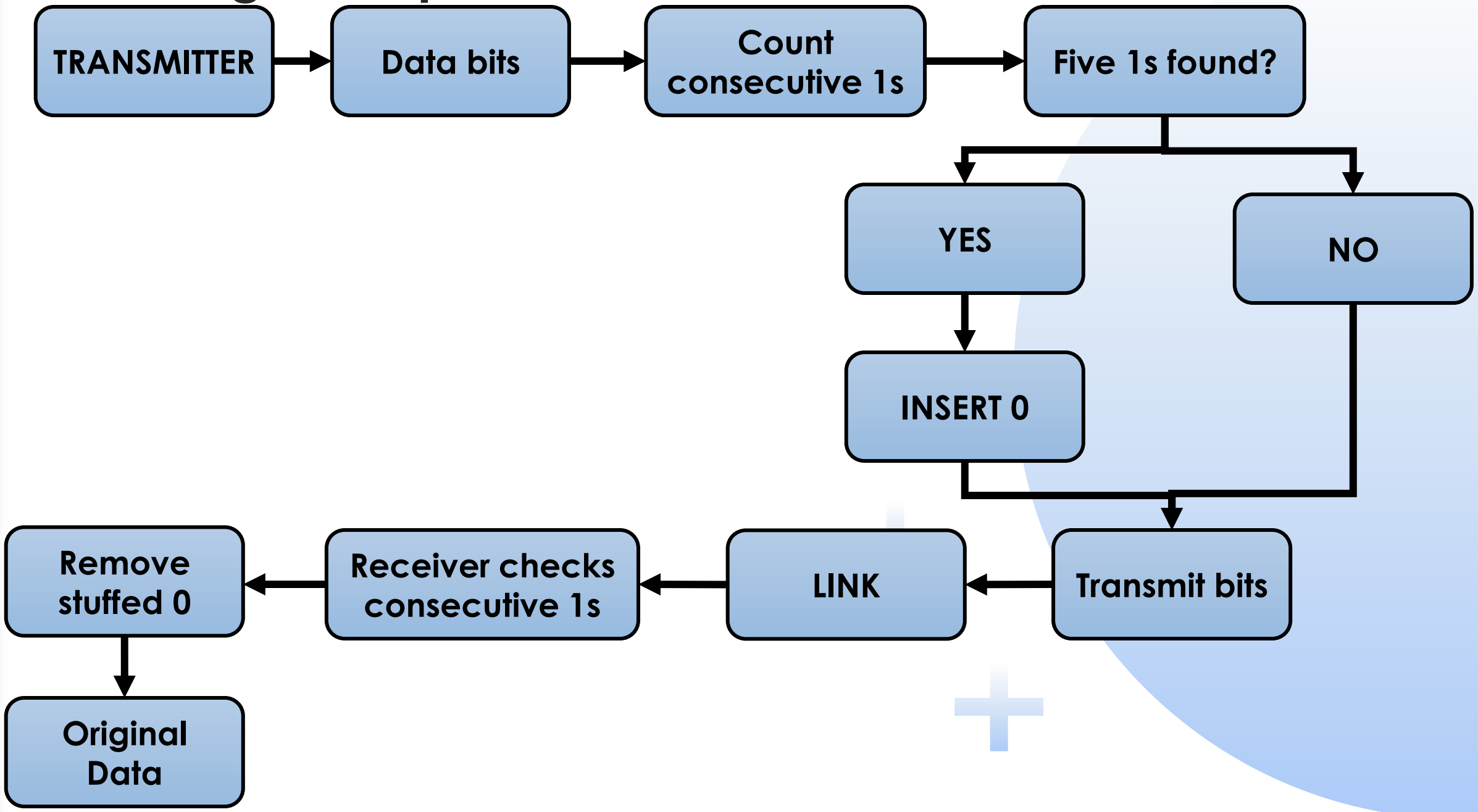
- | 0 | 1 | 1 | 1 | 1 | 1 | 1 | 0 | ➡ **Flag**
- | 1 | 1 | 1 | 1 | 1 | 0 | inside data ➡ remove the **0**

Bit Stuffing and Data Transparency

Data Transparency

- Bit stuffing allows **arbitrary bit patterns** to be carried in the data field.
- The receiver can distinguish:
  - *Frame boundary vs. Data containing similar bit patterns*
- because the stuffing rule prevents the flag pattern from appearing unintentionally within the data.
- **Key idea:**
  - The data can contain arbitrary bit patterns without destroying frame synchronization.

Bit Stuffing: Complete Process



9.5
Comparing the Three



Character Count vs Byte Stuffing vs Bit Stuffing

Feature	Character Count	Byte Stuffing	Bit Stuffing
Boundary method	Count field	Flag byte	Flag bit pattern
Unit	Bytes	Bytes	Bits
Main mechanism	Specify frame length	Insert ESC	Insert 0
Receiver identifies end	From count	From FLAG	From FLAG pattern
Main problem	Corrupted count	FLAG/ESC in data	Stuffed bits increase length
Example	Byte count	Byte stuffing / PPP	HDLC

Why Do We Need Stuffing?

The common problem

- Special patterns are used to identify frame boundaries.
- But the same patterns might naturally occur in the data.

Solution

Modify the transmitted data temporarily.



Byte stuffing



Bit stuffing



Effect of Stuffing on Frame Length

- Stuffing can make the transmitted frame **longer than the original data**.
- The amount of additional data depends on the **contents of the frame**.
- For example:
  - Byte stuffing adds extra bytes when FLAG/ESC occurs.
  - Bit stuffing adds extra 0 bits when five consecutive 1s occur.

E.g.:

- If there are no flag bytes in the data,
  - 100 bytes might be carried in a frame of roughly 100 bytes.
- If, the data consists solely of flag bytes,
  - each flag byte will be escaped and the frame will become roughly 200 bytes long. With bit stuffing, the increase would be roughly 12.5% as 1 bit is added to every byte.

Bit Stuffing in HDLC

HDLC Frame

FLAG	Address	Control	Data	FLAG

FLAG : 01111110

- For bits between the starting and ending flags:
  - **Insert 0 after every five consecutive 1s.**

- **Important**
  - Bit stuffing allows arbitrary bit patterns in the data while preserving the special meaning of the flag.

PPP and Byte Stuffing

- **PPP** is an example of a byte-oriented data-link protocol.
- PPP is **byte oriented**.
- PPP uses **byte stuffing**.
- PPP frames begin with the HDLC flag byte **0x7E**.
- A flag appearing in the payload is escaped.
- **Key comparison**
  - HDLC → bit oriented → bit stuffing
  - PPP → byte oriented → byte stuffing

9.6 Conclusion



One Problem, Three Solutions

Problem

- **How does the receiver know where one frame ends and another begins?**

Solution 1 — Character Count

- COUNT → read specified number of bytes

Solution 2 — Byte Stuffing

- FLAG → use special byte
- ESC → protect FLAG/ESC inside data

Solution 3 — Bit Stuffing

- FLAG = 01111110
- Five 1s → insert 0

Key Takeaways

- Framing divides a continuous bit stream into frames.
- **Character Count**
  - Uses a length/count field.
  - Vulnerable to errors in the count.
- **Byte Stuffing**
  - Uses special flag bytes.
  - ESC protects FLAG and ESC appearing in data.
- **Bit Stuffing**
  - Uses a special flag bit pattern.
  - Inserts 0 after five consecutive 1s.
  - Used with HDLC.
- Stuffing provides **data transparency** while preserving frame boundaries.

Review Questions

Basic Understanding

1. What is framing?
2. Why does the data link layer need framing?
3. What is the purpose of a frame boundary?
4. What are the three framing methods covered in this lecture?
5. How does character count identify the end of a frame?
6. What is the main disadvantage of character count framing?
7. Why can a single error in the count field cause loss of synchronization?
8. What is a flag byte?
9. Why can a flag byte appearing inside data cause a problem?
10. What is byte stuffing?

Review Questions

Character / Byte Stuffing

11. What is the purpose of an ESC byte?
12. What happens when a FLAG byte occurs inside the data?
13. What happens when an ESC byte occurs inside the data?
14. What is destuffing?
15. Why must the ESC byte itself also be stuffed?
16. After destuffing, what should happen to the original data?
17. Why is byte stuffing called a transparent mechanism?

Review Questions

Bit Stuffing

18. What is the HDLC flag pattern?
19. What is the rule used for bit stuffing?
20. What does the transmitter do after detecting five consecutive 1s?
21. What does the receiver do with the stuffed 0?
22. Why does bit stuffing prevent the flag pattern from appearing inside the data?
23. What is meant by data transparency?
24. Why is bit stuffing not tied to 8-bit characters?
25. Which protocol in the books uses bit stuffing?
26. Which protocol in Tanenbaum uses byte stuffing?

Review Questions

Application / Calculation Questions

27. A frame contains the byte sequence: A B FLAG C . Show the transmitted sequence after byte stuffing.
28. A frame contains: A ESC B FLAG . Show the sequence after byte stuffing.
29. Apply bit stuffing to: 0111110
30. Apply bit stuffing to: 111110111111
31. Given the transmitted bit sequence, identify which 0s were inserted by bit stuffing.
32. Compare character count, byte stuffing, and bit stuffing in terms of:
    - Boundary identification
    - Unit of operation
    - Main mechanism
    - Main weakness

Thank you

COM 3401

DATA COMMUNICATION AND NETWORKING

Lecture 09

FRAMING